Processingをはじめよう

第10章

オブジェクト

目次

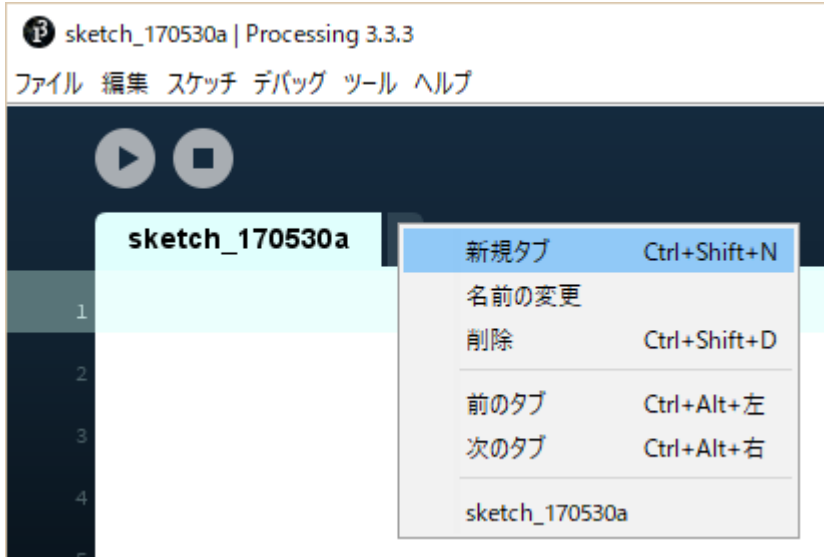
- フィールドとメソッド
- クラスを定義する
- オブジェクトの生成
- タブ機能
- Robot 8: Objects

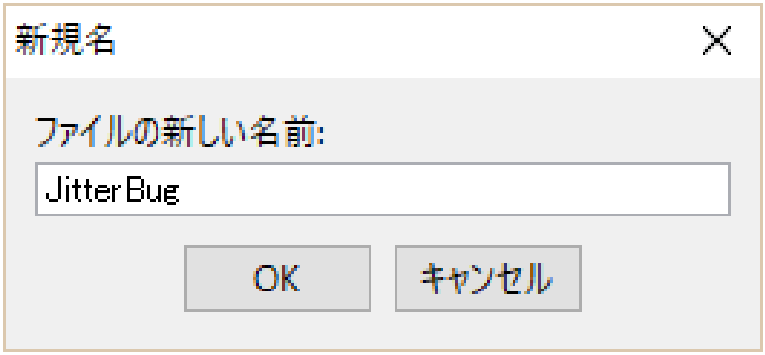
クラスを定義する

- オブジェクト＝もの
  - 関連しあう変数と関数の集合体
    - 変数 ⇒ フィールド、インスタンス変数
    - 関数 ⇒ メソッド
  - データと命令を結び付けたもの
  - 関連しあうデータと、それに働きかける命令や振る舞いを一つにまとめたもの

クラスの定義

- Example8-9 クラスを使って書き換え
- 新規タブを選択、クラスファイルを追加
  - クラス名 JitterBug





Example 8-9

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

クラスの定義

- クラス＝オブジェクトの仕様書
  - 建築に例えると・・・
    - クラス：家の設計図、オブジェクト：家そのもの
- クラスの定義

1. クラスのブロックを作成
2. フィールドを追加
3. コンストラクタの作成とフィールドへの代入
4. メソッドの追加

変更①　クラスのブロックを作成

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

```
// 新規タブ JitterBug

class JitterBug {

}
```

クラスの
ブロック

```
class クラス名 {

}
```

クラス名は大文字で開始する
単語の頭文字を大文字にする

変更②　フィールドの追加

```
float speed = 2.5;
int diameter = 20;
float x;
float y;

void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;
}
```

コンストラクタで
値を代入するフィールドは
フィールドの宣言のみ行う

それ以外のフィールドは
フィールドの宣言と
初期化を行う

```
class クラス名 {
  型 フィールド名;
}
```

変更③-1 コンストラクタの作成

```
void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug(){}
}
```

コンストラクタは
クラス名と同じ名前の
特別なメソッド

```
class クラス名 {
  型 フィールド名;

  クラス名(){}
}
```

変更③-2 フィールドへの代入

```
void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

voi
  x
  y
  el
}
```

```
class クラス名 {
  型 フィールド名;

  クラス名(引数型 引数名){
    フィールド名 = 引数名;
  }
}
```

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
             float tempDiameter    ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

}
```

同じ型

コンストラクタで
値を代入するフィールド

値を代入するための引数

各フィールドに引数を代入する

変更④　メソッドの追加

```
class クラス名 {
  型 フィールド名;

  クラス名(引数型 引数名){
    フィールド名 = 引数名;
  }

  戻り値の型 フィールド名(引数型 引数名){
    文;
    return 式;
  }
}
```

```
void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

draw()内の各処理をメソッドで定義

```
// 新規タブ JitterBug

class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
             float tempDiameter ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move(){}
  void display(){}

}
```

JitterBugが移動するメソッド move()追加

JitterBugを表示するメソッド display()追加

変更④　メソッドの追加

```
void setup() {
  size(240, 120);
  x = width/2;
  y = height/2;
}

void draw() {
  x += random(-speed, speed);
  y += random(-speed, speed);
  ellipse(x, y, diameter, diameter);
}
```

Example 10-1 につづく

```
class JitterBug {
  float x;
  float y;
  int diameter;
  float speed = 0.5;

  JitterBug( float tempX, float tempY,
             float tempDiameter ) {
    x = tempX;
    y = tempY;
    diameter = tempDiameter;
  }

  void move(){
    x += random(-speed, speed);
    y += random(-speed, speed);
  }

  void display(){
    ellipse(x, y, diameter, diameter);
  }

}
```

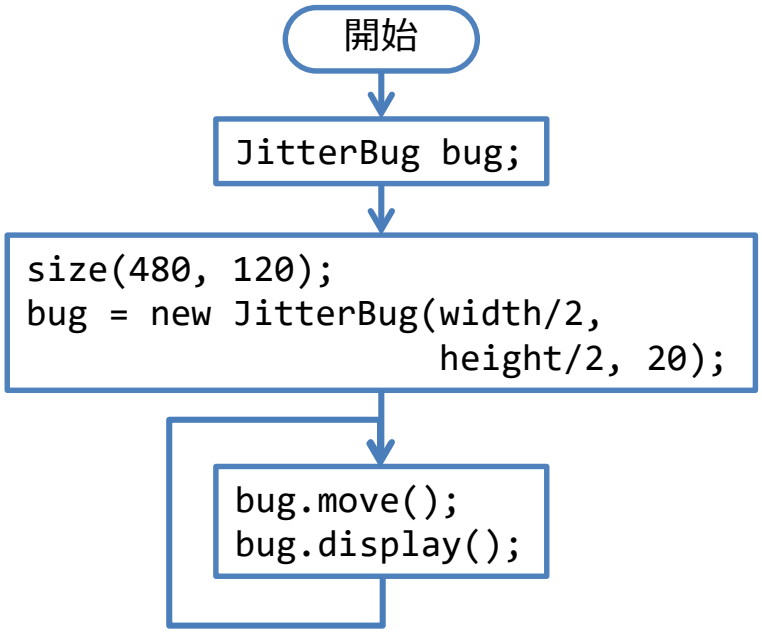
JitterBugが移動するメソッド
move()追加

JitterBugを表示するメソッド
display()追加

オブジェクトの生成

- Example 10-1 オブジェクトを作る

```
JitterBug bug; // オブジェクト変数の宣言

void setup() {
  size(480, 120);
  //オブジェクトの生成
  bug = new JitterBug(width/2, height/2, 20);
}

void draw() {
  bug.move();
  bug.display();
}
```




```
//JitterBug タブ

class JitterBug {

  // 省略

}
```

オブジェクトの生成

- クラスをもとにオブジェクトを作る
  - オブジェクト変数を宣言

```
JitterBug bug;
```

クラス　オブジェクト変数

  - new演算子とコンストラクタでオブジェクトを生成

```
bug = new JitterBug(width/2, height/2, 20);
```

オブジェクト変数　new 演算子　コンストラクタ

- メソッドの実行
  - オブジェクト変数名.メソッド(引数);

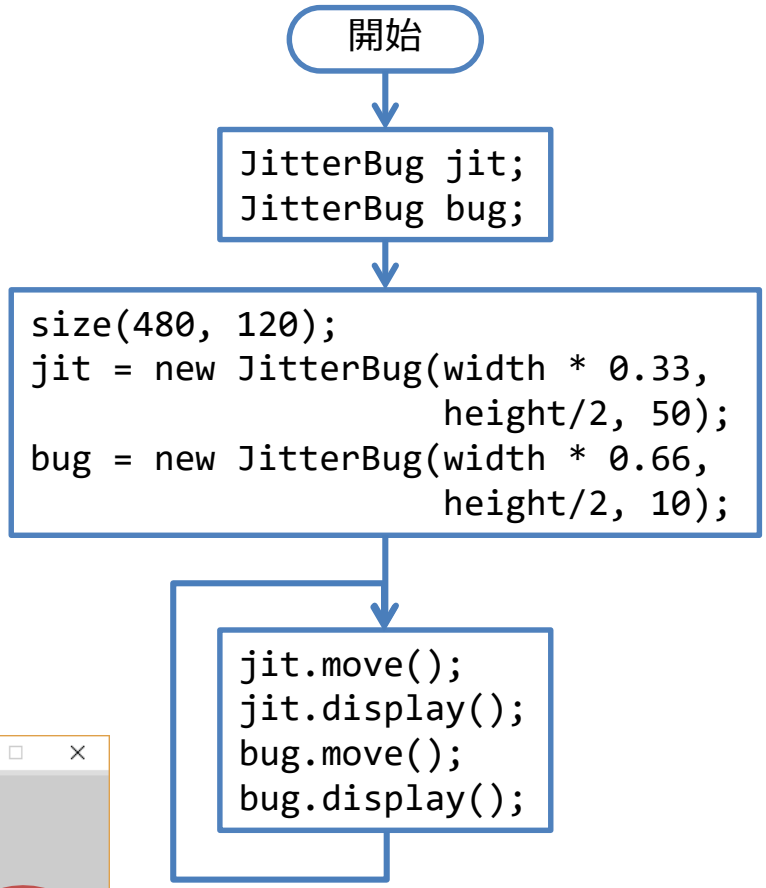
```
bug.move();
```

オブジェクト変数　メソッド　ドット演算子

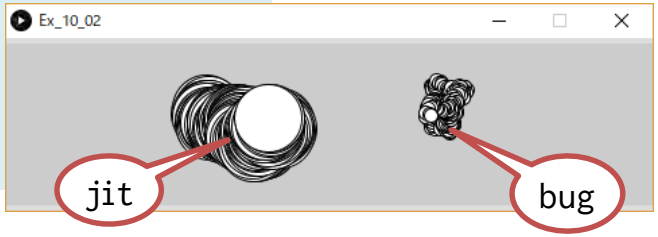
オブジェクトの生成

- Example 10-2 複数のオブジェクトを作る

```
JitterBug jit;
JitterBug bug;

void setup() {
  size(480, 120);
  jit = new JitterBug(width * 0.33, height/2, 50);
  bug = new JitterBug(width * 0.66, height/2, 10);
}

void draw() {
  jit.move();     //jitを移動する
  jit.display(); //jitを描く
  bug.move();     //bugを移動する
  bug.display(); //butを描く
}
```

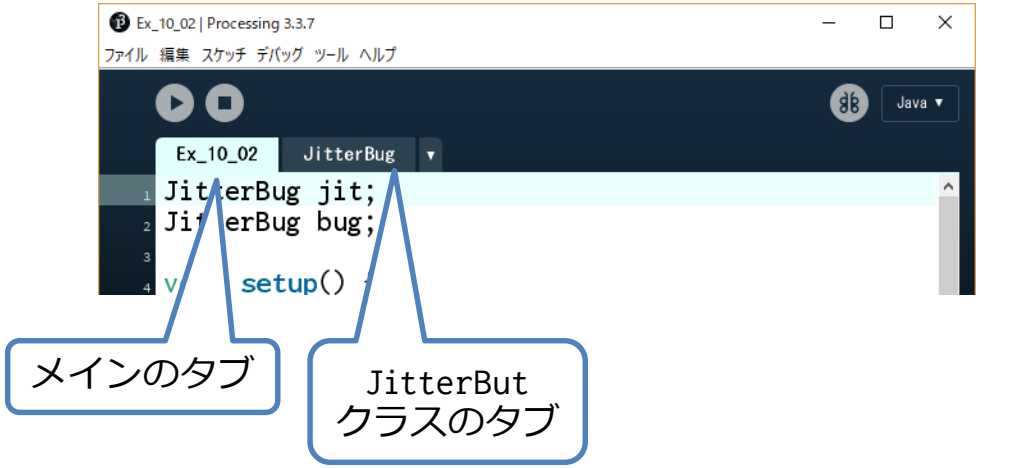


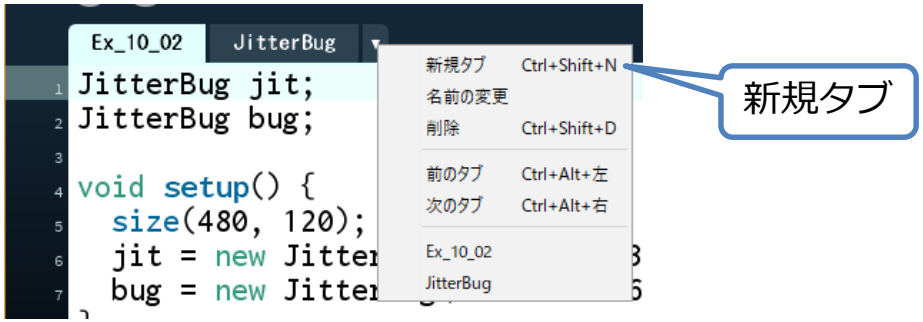

```
//JitterBug タブ（Example 10-1と同じ）

class JitterBug {

  // 省略

}
```



タブ機能

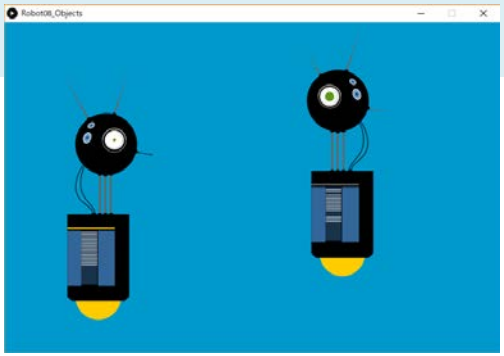
- クラスはメインのタブと分ける
  - クラスのタブは別ファイルに保存
  - 別のプログラムで再利用できる
  - タブのメニューから新規タブを選択してタブを追加できる





Robot 8: Object

```
Robot bot1;
Robot bot2;

void setup() {
  size(720, 480);
  bot1 = new Robot("robot1.svg", 90, 80);
  bot2 = new Robot("robot2.svg", 440, 30);
}

void draw() {
  background(0, 153, 204);

  bot1.update();
  bot1.display();

  bot2.update();
  bot2.display();
}
```




```
class Robot {
  float xpos;
  float ypos;
  float angle;
  PShape botShape;
  float yoffset = 0.0;

  Robot(String svgName, float tempX, float tempY) {
    botShape = loadShape(svgName);
    xpos = tempX;
    ypos = tempY;
    angle = random(0, TWO_PI);
  }

  void update() {
    angle += 0.05;
    yoffset = sin(angle) * 20;
  }

  void display() {
    shape(botShape, xpos, ypos + yoffset);
  }
}
```